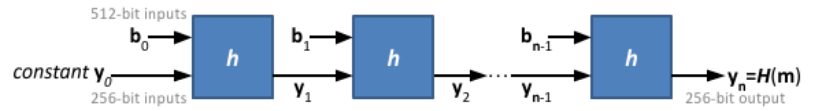


Confidentiality – Message cannot be read by outside parties.
Integrity – Message cannot be edited (without detection) by outside parties (Or message cannot be reconstructed securely)
Authenticity – Message is verifiably from one of the intended parties.
Kerckhoff's Principle – A cryptosystem should remain secure even if attackers know everything but the key.
Crypto Hash – Algorithm that maps data of arbitrary size to string of fixed size. One way, not meant to be decrypted. **NO KEY.**
Length Extension Attack (LE) – Abuses the MD construction and if you're able to find the original pad you can append an arbitrary suffix and construct a verifier such that the server will accept. **Prevented by HMAC because of internal hash and use of key.**
HMAC – Secure verifier. Takes in a key of any length and outputs a fixed digest.
One Time Pad – Combine plaintext with a random key. Impractical because you cannot reuse any part of the random pad which is hard to do. (If you know random key then you can XOR two messages and observe them)
Stream Cipher – Input key into PRG and XOR with message. Never reuse keys or PRG output bits.
Block Cipher – Function encrypts fixed-size (n-bit) blocks with a reusable key. Inverse function decrypts with the same key.
AES – Standard Block Cipher. Fixed block size of 128, key size of 128,192,256. 10,12,14 rounds respectively.
ECB Mode – Encrypt each block independent. Bad because same data will be encrypted the same and thus will be passed on.
CBC Mode – Chains ciphertexts to later ones with a random IV. Must send the IV with ciphertext. Can't encrypt in parallel or out of order.
CTR Mode – Turns block cipher into a stream cipher with a unique nonce and key stream for a particular key. Doesn't require padding and efficient random access but can never reuse the same nonce for the same key.
Malleability – Ability to transform ciphertext into another ciphertext that decrypts to a related plaintext, without knowing the original plaintext.
Cryptographic Doom Principle – If you must perform any cryptographic operation before verifying the MAC on a message you've received, it will somehow inevitably lead to doom.
Diffie-Hellman Key Exchange – Asymmetric Standard for exchanging keys. Primarily used to send private keys over networks.
Forward Secrecy – Adversary cannot read messages in the past, only moving forward.
DH Key Exchange (Achieves Forward Secrecy)
Public parameters: p (large prime), g (primitive root modulo p)
Alice picks some random value between $0 < a < p$ and computes $A = g^a \text{ mod } p$
Bob picks some random value between $0 < b < p$ and computes $B = g^b \text{ mod } p$
Key ($g^{ab} \text{ mod } p$) is shared between Alice and Bob when they compute $A^b \text{ or } B^a$ ($g^{ab} \text{ mod } p$)
Eavesdropping fails, but MITM attack succeeds. Defend with first time trust/digital sigs.
Textbook RSA (Always pad/verify pad otherwise signing is easy to forge.)
Generate large primes p and q, compute $N = pq$, public key (e,N) = (p-1)(q-1) and private key (d,N) where $d = e^{-1} \text{ mod } N$.
Encrypt $c = m^e \text{ mod } N$, Decrypt $m = c^d \text{ mod } N$.
Signature $s = m^d \text{ mod } N$, Verify $m = s^e \text{ mod } N$.
Can be used for Confidentiality, Integrity, and Authenticity
Confidentiality: Public-key encryption
Alice encrypts m using Bob's public key to get c
Bob decrypts c using Bob's private key to get m
Integrity/sender authenticity: Digital signatures
Alice signs m using Alice's private key to get s
Bob verifies m,s using Alice's public key
Both properties: Use both, with two key pairs
Alice encrypts m using Bob's public key to get c, then Alice signs c using Alice's private key to get s
Bob verifies c,s using Alice's public key, then Bob decrypts c using Bob's private key to get m
The only reason we don't use RSA is because it's slow.
HTTP Strict Transport Security (HSTS)
Special header to only use HTTPS. Can be added to a HSTS preload-list to do this auto for first time visits.
CA Weaknesses
Attacker can falsely convince a CA that they control domain and get a cert.
Defend by creating a DNS record and prohibit, admin, administrator, webmaster, hostmaster, postmaster emails from being used.
Multi-perspective validation requires challenge validation verification from several 'locations' in order to prove identity.
Certificate Transparency Log
Server used to monitor certs being issued can be viewed by CAs to ensure.
Five Protocol Layers
Application – DNS, {SSH, FTP, SMTP, NNTP, HTTP} / **Transport** – UDP, {TCP} / **Network** – IP / **Link** – Cellular, Wifi, [Ethernet] / **Physical** – Radio, [Copper, Fiber]
Ethernet – Communicates via packets using 48-bit MAC addresses. Clients can change the MAC arbitrarily. 5 fields, Dest MAC, Source MAC, EtherType (0x0800 IPv4, 0x0806 ARP, 0x86DD: IPv6), Payload (IP, ARP, etc), CRC checksum.
Ethernet Switch learns what MACs are on each port.
IPs (IPv4 = 2^32, IPv6 = 2^128 though uncommon, INSECURE)
IP only provides routing and fragmentation and reassembly. Nothing else.
Sender sets source address, (not downloaded by) 10000074484854 from CourseHero.com on 12-16-2024 20:10:22 GMT -0000
Sender sets destination address. IP HEADERS ARE NOT AUTHENTICATED.

Key Length
Keys should be 128 bits.
256 bits for Quantum Safety.
2^32 can be brute forced.
2^64 debatable
2^128 impossible.

Merkle-Damgard Construction – Pad input m to the next multiple of 512 bits (Collision Res (sha-256) and split into 512-bitblocks. Use IV and then hash consecutively. (Weak against LE)

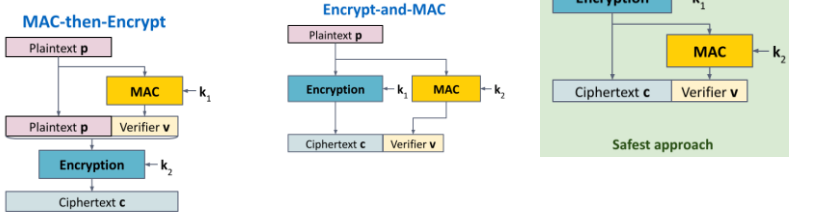


Properties of a Strong Hash Function:

- Preimage Resistance** – Given output h, hard to find an input m such that $h = H(m)$
- Second-Preimage Resistance** – Given input m_1, hard to find different m_2 such that $H(m_1) = H(m_2)$ match.
- Collision Resistance** – Hard to find any pair of inputs m_1, m_2 where $H(m_1) = H(m_2)$ (Strongest form of resistance)
Collision resistance implies second-preimage resistance, but not preimage resistance.
Second preimage attack implies collision attack.
- If message is a multiple of the block size – Add an entire block of padding**
- Padding Oracle Attack** is a MITM attack where we abuse the CBC **Malleability** to change cipher-text until we get an error that lets us decipher details about the plaintext. This is made primarily due to the **MAC-Then-Encrypt** nature taken in P1. (MAC error // Padding Error)
- CTR Mode Malleability** allows you to directly flip bits in the corresponding plaintext block.
- Malleability** necessarily means that integrity is not ensured.

Two Approaches to Authenticated Encryption

Generic Composition (Encrypt-Then-MAC, AEAD)
All-In-One Primitive (GCM)



Bleichenbacher Attack

Small e attack, but you add 1 because the padding isn't checked carefully enough

Attacks on RSA

Small e attack, if e is < 3 then cube root is fast. And doesn't wrap around.

Ciphertext malleability

Key generation failures

Forging signatures for random/specific messages

Safely encrypting with RSA

Use a PRF to derive a key from x.

Generate random $x < N$

$c1 = x^e \text{ mod } N$

$k = \text{sha-256}(x)$

$c2 = \text{AES}_k(m)$

Toy Secure Channel

- DH to generate shared key
- RSA signature to confirm identity (authentication)
- PRF to generate symmetric keys
- AEAD or Encrypt-Then-MAC for messages (Integrity/Confidentiality)

Routers

Devices with multiple link layers. Communicate with BGP (Border Gateway Protocol) which has ISPs announce their networks and connectivity. Helps with routing global.

Ports (Ports numbered 1-65535). Common Ports:

80 HTTP, 443 HTTPS, 25 SMTP, 67 DHCP, 22 SSH, 23 Telnet.

UDP - Faster transport-layer protocol. Wrapper around IP that adds multiplex traffic for an application. Connectionless, out of order data, no flow control, no congestion control, no security. It's used for the speed. Used by DNS, QUIC, games and voice/vid.

TCP - General transport-layer protocol. Two data streams once in each direction with buffers. Data received in order, has congestion control. Doesn't provide strong security.

TCP handshake includes client sending SYN with seq, random 3 byte int, server sends own seq back and ack which was previous seq + 1 and then client sends ack and seq + 1

Network Attackers

Off-Path – Network participant. Can talk to hosts but cannot see packets. Weakest.

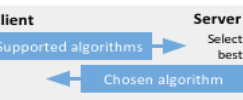
On-path – Sees copy of victim's packets. Can add but can't change/block.

In-path (MITM) – Can see, add, change, block victim's packets. Strongest. Ways to become this attacker via ARP spoofing, BGP hijacking, DNS attacks.

TLS Protocol Handshake



Negotiate Crypto Algorithms



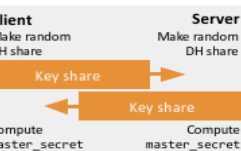
Find best mutually supported:

Key exchange algorithm
e.g., EC DHE w/ Curve25519

Signature algorithm
e.g., RSA w/ SHA-256, PKCS #1 pad

Symmetric crypto algorithm
e.g., AES-256

Establish Shared Secret



Mutually compute a **shared secret**.

Diffie-Hellman for **forward secrecy**.

Derive **symmetric keys** from shared secret and encrypt and integrity check all further data

Is it the real server or an active attacker?

Authenticate the Server



Server signs, and client verifies, a hash of the entire **handshake transcript** to this point.

How does client authenticate the server's public key?
(Find out in a few slides)

DNS (Domain Name System) Resolves hostnames to IP addresses **google.com to IP Recursive Servers** – Answers queries from clients by asking other servers. (1.1.1.1)
Auth. Servers – Domain’s own server. Gives definitive answers for parts of namespace
DNS Zones (example query of www.google.com)
Client asks ISP’s recursive DNS server for www.google.com and the root DNS server responds with .com’s namespace and then .com gives google.com’s namespace and google.com is the auth. Server and responds with www.google.com.
Control over each region of namespace. Root zone contains all TLDs controlled by 13 auth servers.
DNS Packets (Uses UDP packets for speed)
Broken up into questions, answers, authority, and additional records.
DNS Hijacking – Attacker hacks into home router changes DHCP settings to attacker-controlled DNS server so it can inject malware.
DNS monitoring and blocking – ISP monitors DNS reqs to track users and injects fake resps. to block particular websites. **Off-path DNS cache poisoning.**
DNSSEC – Auth DNS servers sign DNS resp. with crypto key. Clients verify a resp. legit by checking sig. through PKI like HTTPS. Provides (AI) not (C), Resp. in clear.
DNS-over-TLS/HTTPS – DNS through TLS/HTTPS instead of UDP. Provides (CI). Does not provide (A). **Vulnerable to Downgrade attacks.**
Modern Thinking –
DoS attacks stems from asymmetry (Response cost > Query cost) Occurs all layers
TCP SYN Flood Attack – Attacker sends SYN packets from many spoofed source IPs
Defense: Syn cookies which MAC syn request and only stores in queue if succeed.
Amplification – Send small UDP packet with large response payload. (ANY, MONLIST). Can be done from many IPs. Hence, this is a **DDoS**. **Botnets** are used in DDoS which are any compromised machines controlled by someone.
Defenses: Ingress Filter: Restrict packets by region, but little incentive so pointless.
Content Delivery Networks(CDN): Comp. like Cloudflare absorbs DDoS by providing bandwidth to victim. Then uses security team to filter.

HTTP Protocol	URLS
GET (Retrieve Data: Shouldn’t change server state)	Host: Server’s domain name or IP
POST(Can submit data: Causes change or effect)	Port: TCP port (443 - HTTPS, 80 - HTTP)
Same-Origin Policy (SOP)	Path: Identifies resource to server
Web page from one host should not be able to read or modify content from another host.	Query: Parameter passed to server
Cookies Scope	Fragment: Only visible to client(?)
DOM: Scoped based on (scheme, host, port)	Cookies
Cookies: Scoped based on ([scheme], domain*)	Piece of data a server sends to the browser.
Setting a Cookie:	Sometimes stored and returned later.
A site can set a cookie for its own domain or any parent domain, as long as parent domain is not a public suffix (.com, .gov, and countries).	Useful for:
Reading a Cookie:	Maintaining session state
A site can read cookies set for its own domain or any parent domains.	Personalization
CSRF Attacks (Cross-site request forgery)	Tracking
Cause the user to perform unwanted actions on behalf of the attacker. Relies on cookies on any req.	Clients can read, change or erase cookie data, so they need to be stored
Two Main Types	cryptographically or randomly make a value so that it’s tied to server database
Login Attack – Log victim’s browser into honest site with account controlled by the attacker	Cookies Weaknesses
CSRF via POST request – Attacker triggers a POST request using HTML + JS.	Cookies will be sent over HTTP if they sent over HTTPS
CSRF exploits the trust that a site has in a user’s browser. Users visit a malicious website.	Server can set ‘Secure’ attribute to cookies it assigns to prevent this.
CSRF attacks target state-changing request, not data theft, since attacker cannot see the response	Cookies can also be ready by any JS running the origin.
XSS (Cross Site Scripting) (Attacks Browsers)	Attribute ‘HttpOnly’ prevents the cookie from being accessed by the DOM.
Two types (Takes place Client-Side)	Cookies are commonly auth tokens.
Reflected XSS (Project 2) (One User At A Time)	CSRF Defenses (CSRF relies on cookies)
Echos script back to same user in context of the site	SameSite cookie attribute
Stored XSS (All Users)	Refer validation (check site come from)
Stores malicious code in to target other users on resource managed by the server, such as a db.	Secret token validation (SOP prevent grab)
Certificates	SameSite=Strict
A message asserting the server’s identity and its public key, signed by CAs.	Cookie isn’t sent in any cross-site context
Certificate Authorities (CAs)	SameSite=Lenx
Someone trusted by one or more users as an authority in a network that issues, revokes, and manages digital certificates.	Cookie is sent when navigating to cross-site links, but not on cross-site subrequests.
Intermediate CA Certs	XSS Defenses
Lend permission to sign further certs. Used to Delegate trust to other CAs and use a separate key From long-term root key stored offline for safety.	Input Validation – Check everything against what should be allowed.
SSL stripping	TLS - Cryptographic protocol layered above TCP to provide a secure channel.
MITM attack to force HTTP on website (legacy)	HTTPS = TLS + HTTP
SQLi - Abuses SQL syntax to inject commands into the database. Defend by:	TLS achieves C/I – AEAD ciphers achieves A with public key crypto and CAs
Parameterize SQL statements and ORM	After Key share, TLS is safe.
	Root CA - The CA whose public key is used to authenticate all certificate chains within that community.
	Obtaining a Cert
	Server must prove identity to CA via Email, DNS, or HTTP on local network.
	ACME automates this
	Injection Attack – Program mistakes data for code
	Attacks Databases, takes place Server-Side

Control Hijacking
Binary exploitation is the process of subverting a compiled application such that it violates some trust boundary in a way that is advantageous to you, the attacker.
General Purpose Registers
EAX,EBX,ECX,EDX,EDI,ESI
Special Purpose Registers
EIP – Instruction pointer
ESP – Stack pointer
EBP – Frame/Base Pointer
Stack
Stack starts at highest memory address (0xffffffff and goes up to 0x00000000)
pop eax replaces eax with top of stack.
Frame Pointers – Point to bottom of ‘current’ stack frame.
Stack Pointers – Point to arbitrary memory location depending on control flow.
Instruction Pointer – Point to next instruction to be executed.
Stack Order
Local Variables, function being called’s args, return address, caller’s frame pointer, then repeat.
Buffer Overflow
Buffer allocated gets something larger than anticipated and overwrites the stack.
Shellcode
Compiled x86 code injected into the application to jump to our arbitrary instructions.
Forbidden Characters
strcpy() – 0x00
gets() – “\n”
scanf() – Any whitespace.
Hard to Guess Addr
Noop sled – Input a bunch of noops into the buffer and guess a good, estimated guess of the return addr and let it slide to your shell code.
DEP
DEP prevents injected shellcode.
Data-Only Attacks
Buffer overflow local variables to change behavior.
Return to Libc
Common counter to DEP, reuse code that already marked as executable.
Invoke any finction that exists in the binary. Often try:
Execv(“/bin/sh”, 11, addr of first arg)
Extraneous Function Removal
Remove all functions we don’t need. (Hard to do reliably)
ROP
Small section of code contains a very small number of instructions, not an existing function body.
Build arbitrary functionality via gadgets. Basically can do whatever you want given enough stuff. Gadgets must end in a **ret**.
ASLR
Move around where the code lives. Randomize the address space. Make it really hard to predict references to a particular address.
Can be defeated by Over-read a single pointer in libc, ‘finding it’. Then you lineup the code and find the offset, however if ASLR is **Fine-Grained** (shuffling within code chunks) then it’s impossible.
Stack Canaries
Push a ‘canary’ basically a password onto the stack and check if it’s not saved properly.
Buffer Over-Read
Attack can probe the server to find the 2024-10-12 12:00:00 GMT+00:00 Continue unphased. (Heartbleed)
Three Ways Authenticate
Something you **know, have, are.**

Integer Overflow
Defender chooses to use strncpy, snprintf, fgets, etc. Can overflow integer to get it wrap around or do malicious actions.
Principle of Least Privilege:
Every program and user should operate using the least amount of privilege necessary to complete its job.
Access Control - Security Model Subjects (Who) – UNIX: Users
Objects (What) – UNIX: Files, devices, pro
Operations – Ways subject can operate on objects (read,write, call).
Principle of Complete Mediation
Every access to every object must be checked by authority by a mediator. (DO NOT CACHE CHECKS. Vulnerable to Time-Of-Check, Time-Of-Use Vulnerabilities).
Confused Deputy Problem (ex. CSRF)
Low-privilege process tricks high-privilege process into performing an action it couldn’t perform itself.
Confinement – Ensure misbehaving process cannot harm rest of system.
Reference Monitor – Mediates requests from applications. Must **always** be invoked, must be **tamperproof** (cannot be killed unless monitored process is also). Must be small enough to be analyzed and validated. Part of a trusted computing base.
System Call Interposition
Monitor app’s system calls and block authorized calls.
Implemented via ptrace on processes, where it wakes up when the process makes a system call. If the monitor checks policy and it violates it kills the application. Hard to know what a program needs to do always and where it needs to look, so usually not used.
Containers – Virtual Machines
VMs isolate operating systems on a single machine, emulate entire computers and doesn’t know that it’s being used on the Host OS. Mediated by a VMM, aka Hypervisor or Docker.
VMM Security Assumptions
Malware can infect guest OS and guest apps but cannot escape. Cannot infect host OS. Cannot infect other VMs on the same hardware.
Covert Channels – Unintended communication channel between isolated components. Can leak classified data from secure component to public component. The problem is since the VMM is run on the same host OS, you can add a listener for CPU activity, which allows attacker to send messages.
Integrity - Outcome matches voter intent.
Ballot Secrecy
Weak – no one knows how you voted
Strong – even if you try to prove it
Voter Authentication
Only authorized voters can vote
Voters only vote the permitted times
Enfranchisement
Auth’d voters have the opportunity
Availability
System can accept all votes on time
Produces results in a timely manner
Threat Model

- Fucking massive
- Low standardization
- Needs results by election night

Differential Privacy
Adding random noise during data aggregation upsets the statistical accuracy but provides shield for privacy. Works well.